Karishma Premnath
EECE 7205, Stacks and Queues, In-Class Work
Q1 Expression Evaluation Using Stacks

(a) Functions

- `isNumber(t)` a token is a number if it is all digits, or a minus sign followed by digits. This is how `-7` is read as a value and not as subtraction.
- `precedence(op)` `^` is 3, `*` and `/` are 2, `+` and `-` are 1.
- `applyOp(a, b, op)` does one operation on two integers, with `a` as the left operand. Division calls `floorDiv`.
- `floorDiv(a, b)` rounds down toward negative infinity by subtracting one when the signs differ and the division is not exact.
- `evaluate(tokens)` pushes numbers on the values stack, and before pushing an operator it applies every waiting operator that must go first.
- `IntStack` and `CharStack` the two stacks, one holding `int` values and one holding `char` operators.

The pop rule inside `evaluate`:

Top of operators stack is	Then
higher precedence	apply it before pushing
equal precedence, new operator is `+ - * /`	apply it (left associative)
equal precedence, new operator is `^`	leave it (right associative)
lower precedence	leave it

(b) Evaluate $8 + 3 * 2 \wedge 2 - 20/4$

Step	Operator	Calculation	Expression left
			`8 + 3 * 2 ^ 2 - 20 / 4`
1	`^`	$2 \wedge 2 = 4$	`8 + 3 * 4 - 20 / 4`
2	`*`	$3 \times 4 = 12$	`8 + 12 - 20 / 4`
3	`/`	$20/4 = 5$	`8 + 12 - 5`
4	`+`	$8 + 12 = 20$	`20 - 5`
5	`-`	$20 - 5 = 15$	`15`

Step 1 goes first because `^` has the highest precedence. Steps 2 and 3 come next and are taken left to right, then steps 4 and 5 the same way.

Answer: 15

(c) Trace

The two stacks after each token is processed:

Token	Action	Values (bottom to top)	Operators
`8`	push 8	8	
`+`	push `+`	8	`+`
`3`	push 3	8 3	`+`
`*`	push `*`	8 3	`+ *`
`2`	push 2	8 3 2	`+ *`
`^`	push `^`	8 3 2	`+ * ^`
`2`	push 2	8 3 2 2	`+ * ^`
`-`	apply `^`, `*`, `+`, then push `-`	20	`-`
`20`	push 20	20 20	`-`
`/`	push `/`	20 20	`- /`
`4`	push 4	20 20 4	`- /`

At token `-`, three operators are applied because `-` is weaker than all three waiting on the stack:

Apply	Calculation	Values after
`^`	$2 \wedge 2 = 4$	8 3 4
`*`	$3 \times 4 = 12$	8 12
`+`	$8 + 12 = 20$	20

Remaining calculations once the tokens are finished:

Apply	Calculation	Values after
`/`	$20/4 = 5$	20 5
`-`	$20 - 5 = 15$	15

Result: 15

(d) Tests

Expression	Answer	Wrong answer if mishandled
`2 ^ 3 ^ 2`	512	64
`-7 / 2`	-4	-3

$2 \wedge 3 \wedge 2$ gives **512**. When the second `^` arrives, the first one has equal precedence, and `^` is right associative, so it is left waiting instead of being applied. That makes $3 \wedge 2 = 9$ run first, then $2 \wedge 9 = 512$. Treating `^` as left associative would give $(2 \wedge 3) \wedge 2 = 64$.

$-7/2$ gives **-4**. The exact value is -3.5. C++ integer division cuts the fraction off toward zero and gives -3, but the question asks for rounding down toward negative infinity, which is -4. `floorDiv` subtracts one because the signs differ and the division is not exact. A clean division such as $-20/4$ is untouched and stays -5.

Code

```
// Q1: infix evaluation with two stacks, one for integer values, one for operators.

#include <iostream>
#include <string>
#include <vector>
using namespace std;

const int CAPACITY = 100;

class IntStack
{
private:
    int data[CAPACITY];
    int top = -1;
public:
    bool isEmpty() const { return top == -1; }
    bool isFull() const { return top == CAPACITY - 1; }
    int size() const { return top + 1; }

    bool push(int x)
    {
        if (isFull()) { cout << "push " << x << ": overflow" << endl; return false; }
        top++;
        data[top] = x;
        return true;
    }
    bool pop(int &x)
    {
        if (isEmpty()) { cout << "pop: underflow" << endl; return false; }
        x = data[top];
        top--;
        return true;
    }
    bool peek(int &x) const
    {
        if (isEmpty()) return false;
        x = data[top];
        return true;
    }
};

class CharStack
{
private:
    char data[CAPACITY];
    int top = -1;
public:
    bool isEmpty() const { return top == -1; }
    bool isFull() const { return top == CAPACITY - 1; }
    int size() const { return top + 1; }

    bool push(char c)
    {
        if (isFull()) { cout << "push " << c << ": overflow" << endl; return false; }
        top++;
        data[top] = c;
        return true;
```

```
    }
    bool pop(char &c)
    {
        if (isEmpty()) { cout << "pop: underflow" << endl; return false; }
        c = data[top];
        top--;
        return true;
    }
    bool peek(char &c) const
    {
        if (isEmpty()) return false;
        c = data[top];
        return true;
    }
};

bool isNumber(const string &t)
{
    int start = (t[0] == '-' && t.size() > 1) ? 1 : 0;
    for (int i = start; i < (int)t.size(); i++)
        if (t[i] < '0' || t[i] > '9') return false;
    return true;
}

int toInt(const string &t)
{
    bool neg = (t[0] == '-');
    int v = 0;
    for (int i = (neg ? 1 : 0); i < (int)t.size(); i++)
        v = v * 10 + (t[i] - '0');
    return neg ? -v : v;
}

int precedence(char op)
{
    if (op == '^') return 3;
    if (op == '*' || op == '/') return 2;
    return 1;
}

int floorDiv(int a, int b)
{
    int q = a / b;                              // '/' truncates toward zero
    if (a % b != 0 && (a < 0) != (b < 0)) q--;  // step down to floor
    return q;
}

int applyOp(int a, int b, char op)
{
    if (op == '+') return a + b;
    if (op == '-') return a - b;
    if (op == '*') return a * b;
    if (op == '/') return floorDiv(a, b);
    int p = 1;
    for (int i = 0; i < b; i++) p *= a;
    return p;
}

void applyTop(IntStack &values, CharStack &ops)
```

```
{
    char op;
    int a, b;
    ops.pop(op);
    values.pop(b);                                  // right operand comes off first
    values.pop(a);
    values.push(applyOp(a, b, op));
}

int evaluate(const vector<string> &tokens)
{
    IntStack values;
    CharStack ops;
    char topOp;

    for (int i = 0; i < (int)tokens.size(); i++)
    {
        if (isNumber(tokens[i]))
        {
            values.push(toInt(tokens[i]));
            continue;
        }
        char op = tokens[i][0];
        while (ops.peek(topOp) && (precedence(topOp) > precedence(op) ||
                                   (precedence(topOp) == precedence(op) && op != '^')))
            applyTop(values, ops);
        ops.push(op);
    }
    while (ops.peek(topOp))
        applyTop(values, ops);

    int result;
    values.pop(result);
    return result;
}

int main()
{
    vector<string> e1 = {"8", "+", "3", "*", "2", "^", "2", "-", "20", "/", "4"};
    vector<string> e2 = {"2", "^", "3", "^", "2"};
    vector<string> e3 = {"-7", "/", "2"};

    cout << "(b) 8 + 3 * 2 ^ 2 - 20 / 4 = " << evaluate(e1) << endl;
    cout << "(d) 2 ^ 3 ^ 2              = " << evaluate(e2) << endl;
    cout << "(d) -7 / 2                 = " << evaluate(e3) << endl;
    return 0;
}
```

Output

```
(b) 8 + 3 * 2 ^ 2 - 20 / 4 = 15
(d) 2 ^ 3 ^ 2              = 512
(d) -7 / 2                 = -4
```